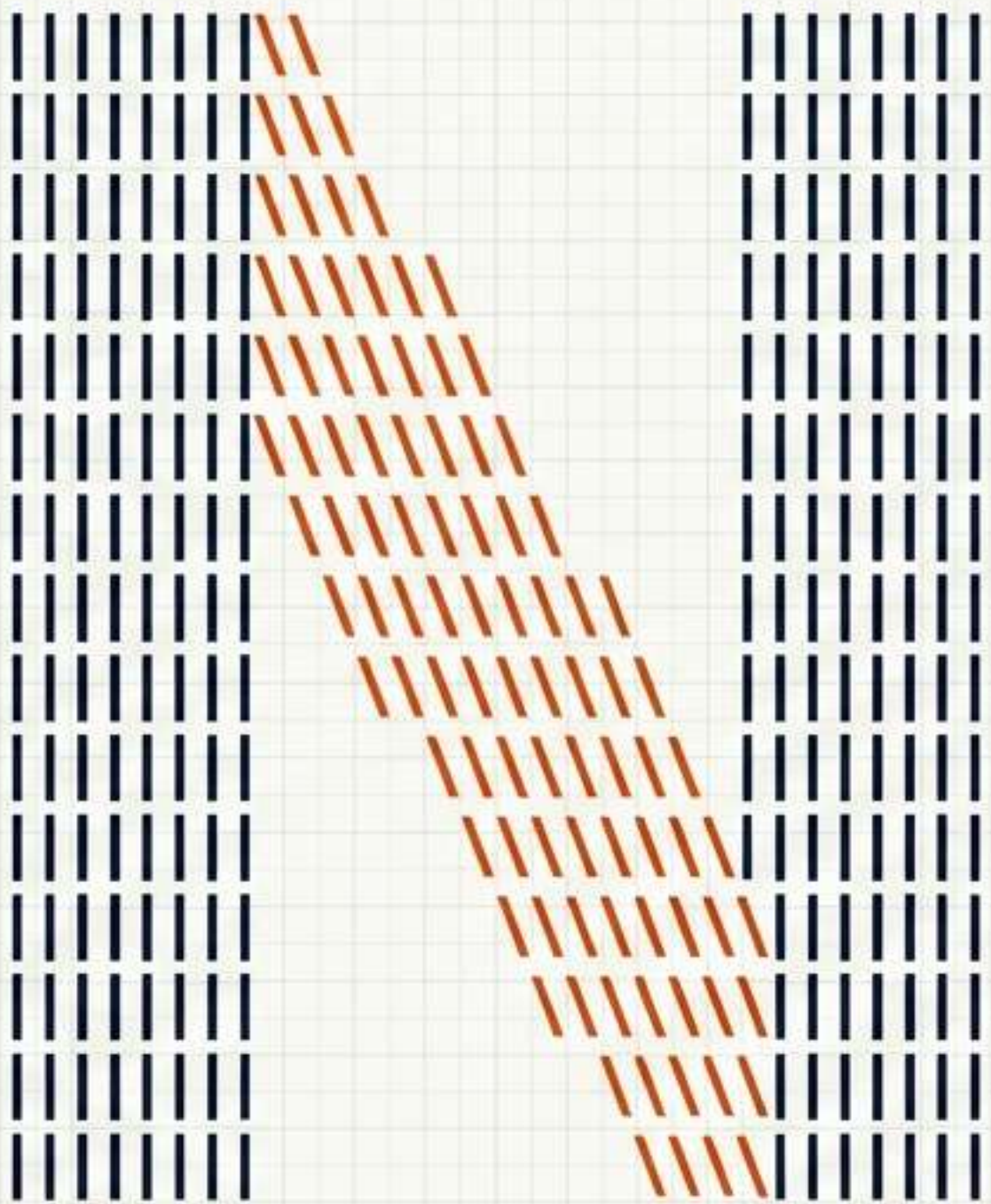




Algorithmic Pattern Design: The N-Shape Logic

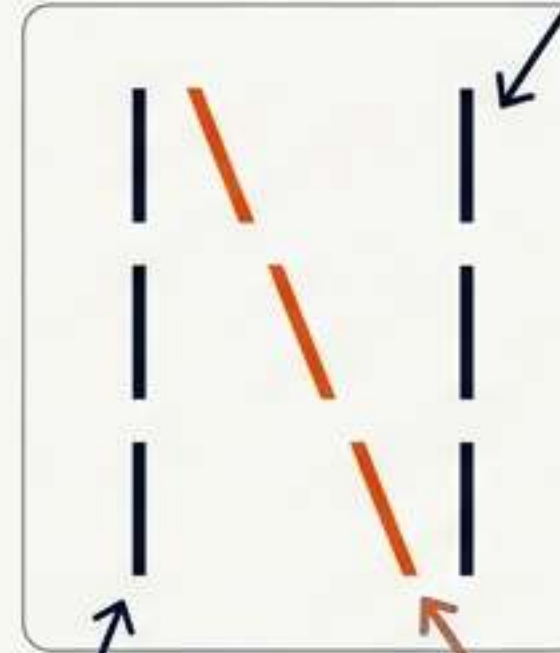
Translating integer input into geometric character output using coordinate logic.

The Blueprint: Requirements & Constraints

INPUT

n = 3|

OUTPUT



Static Elements:
Vertical Borders (|)

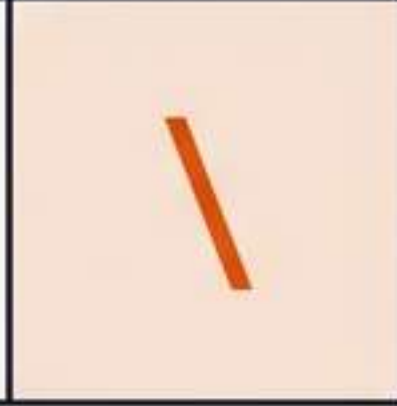


Constraint: No trailing spaces. Line ends immediately after the right bar.

Static Elements:
Vertical Borders (|)

Dynamic Element:
Moving Backslash (\)

Visual Deconstruction: The Matrix View

		0	1	2	
0					
1					
2					

Static Elements: Every row begins and ends with a pipe character |.

Dynamic Element: The backslash \ shifts position in every row.

Observation: Every row tells the same story, but the diagonal moves.

Deriving the Logic: The Identity Rule

	0	1	2
0			
1			
2			

Row 0 = Col 0

Row 1 = Col 1

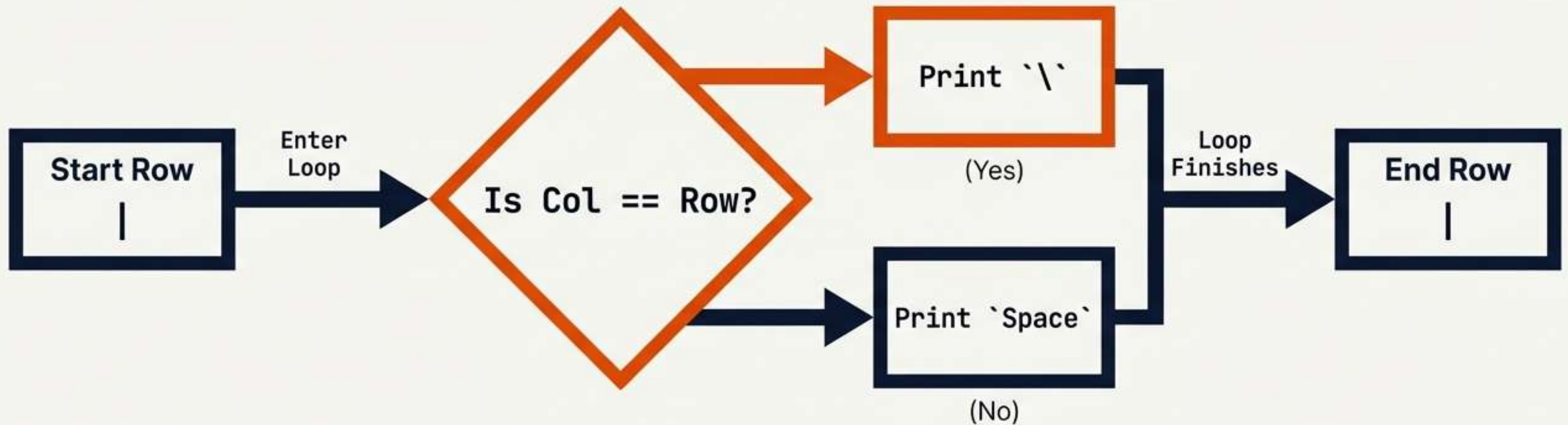
Row 2 = Col 2



The Formula: `if row_index == column_index:`

This is the visual representation of the Identity Matrix concept.
The diagonal element exists only where $i == j$.

Anatomy of a Single Iteration



The logic loop is sandwiched between two static print statements. Inside the loop, we ask a single question for every column.

The Syntax Trap: Escaping the Backslash

The Trap

```
print("\\")
```

Syntax Error ✖

Python interprets `\\` as an escape starter (like `\\n`).

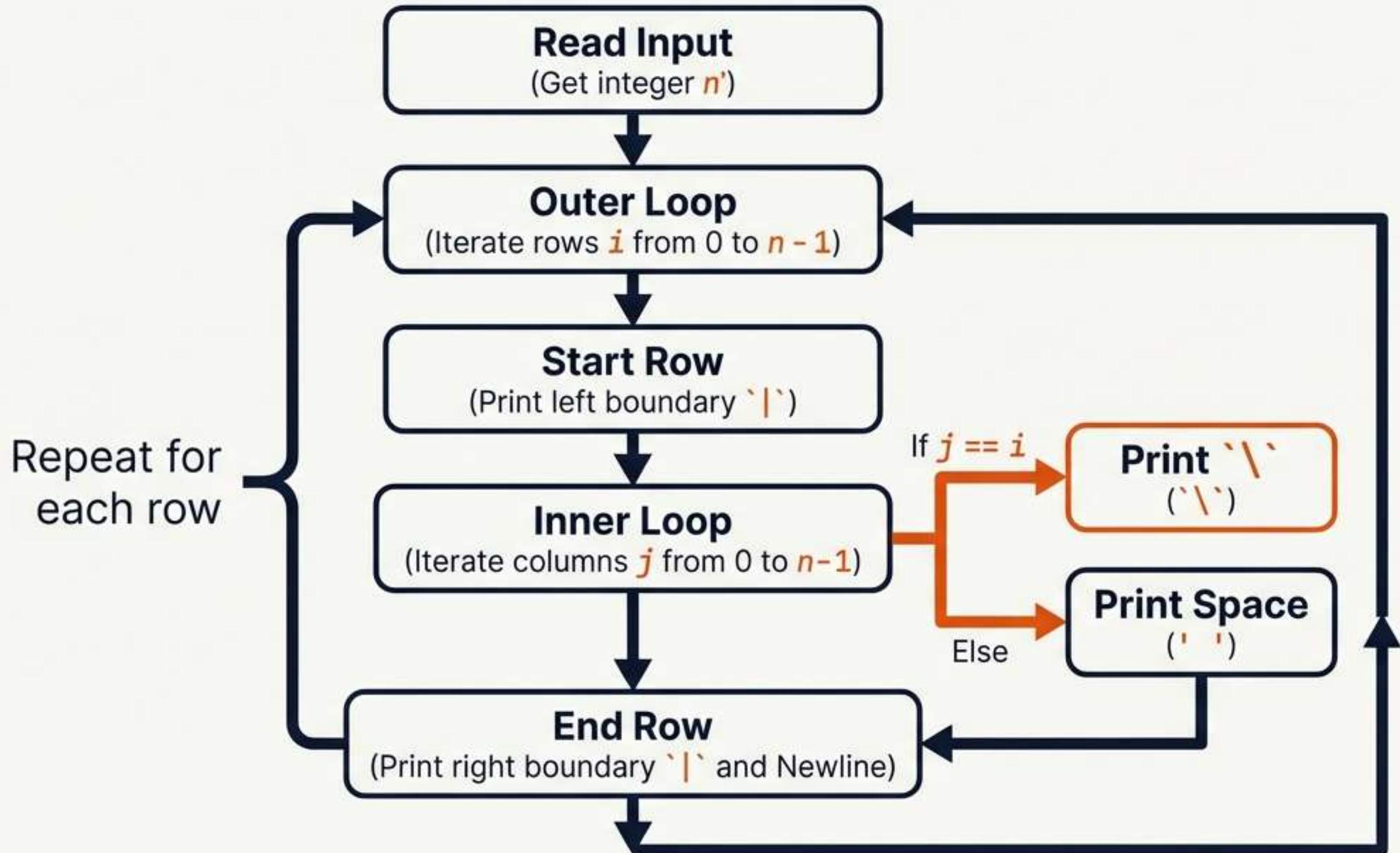
The Escape

```
print("\\\\")
```

The first backslash escapes the second, resulting in a literal `\\`.

To print a literal backslash, you must type **two**. The first one **escapes the second one**.

Developing the Algorithm



Edge Case Verification: $n = 1$

$n = 1$



Row 0, Col 0. Match found immediately. Bars touch the slash.

$n = 2$



When $n = 1$, the 'space' inside the N collapses. The logic ($i = 0, j = 0$) still holds true, printing the diagonal instantly between the borders.

Python Implementation



```
n = int(input())
```

```
for i in range(n):
```

```
    print("|", end="")
```

Left Boundary

```
    for j in range(n):
```

```
        if j == i:
```

```
            print("\\", end="") # The Diagonal (Escaped)
```

```
        else:
```

```
            print(" ", end="") # The Space
```

```
    print("|")
```

Right Boundary & Newline

Prevents
automatic
newlines

Execution Trace (n = 2)

Logic Flow (i = 0)

1. Print `|`
2. j=0 (Matches i=0?) → YES → Print `\\`
3. j=1 (Matches i=0?) → NO → Print `Space`
4. Print `|`

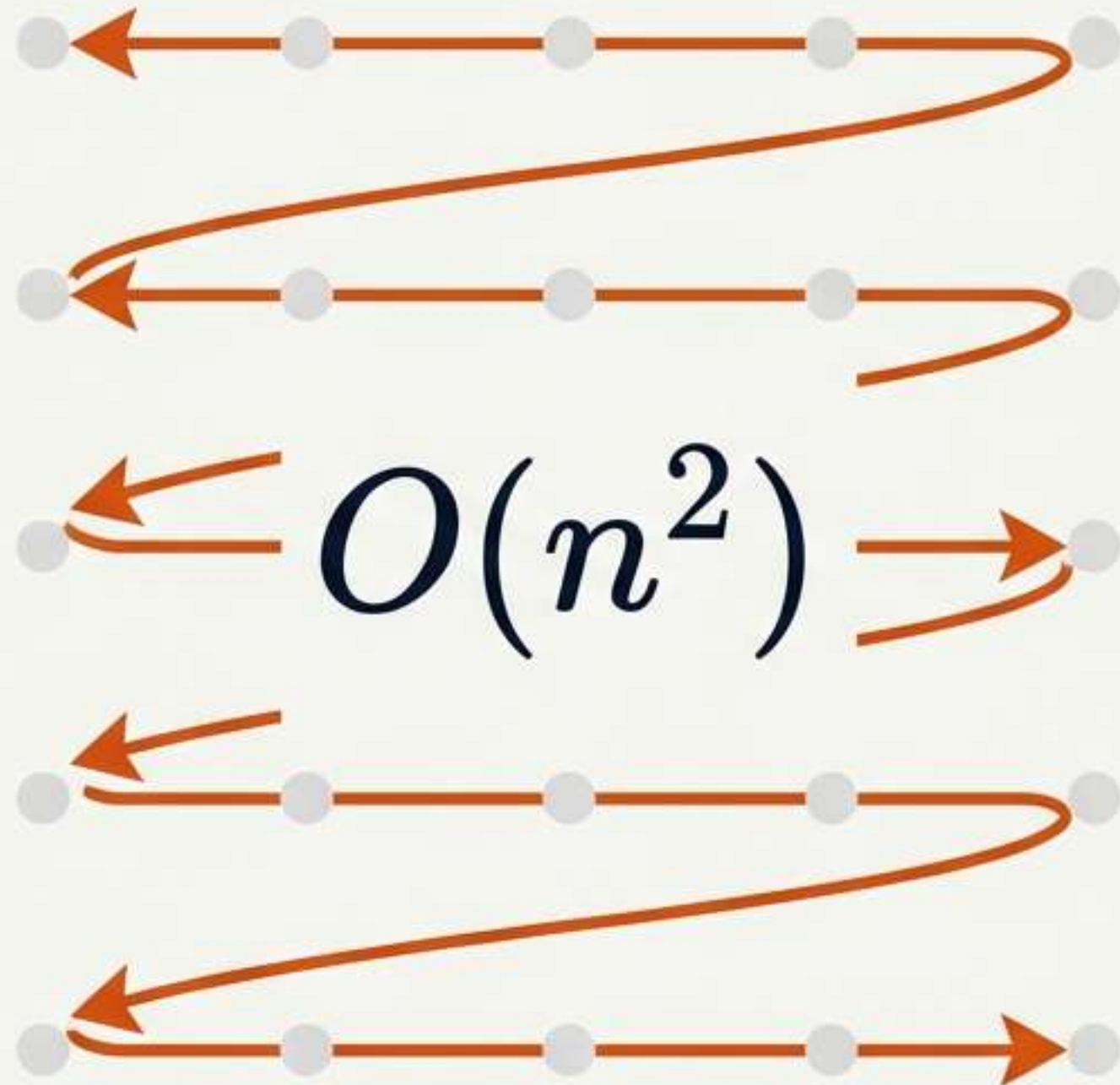
Cycle 1 (i=0): The diagonal matches at the first position. The rest is space.

Console Output



```
| \ |  
|  |
```


Time Complexity Analysis



- **Structure:** Nested Loops (Outer loop runs n times $\times$ Inner loop runs n times).
- **Context:** Since we must visit and print every individual cell in the $n \times n$ grid to check for the diagonal, this quadratic complexity is unavoidable and optimal.

The Pattern Printing Toolkit

- ✓ **Coordinate Mapping:** Always view patterns as a matrix ($\text{\$row, col\$}$).
- ✓ **The Identity Rule:** Use `row == col` to locate the main diagonal.
- ✓ **Syntax Awareness:** Remember ``\`` for backslashes.
- ✓ **Boundary Management:** Handle static borders (`|``) outside the inner loop.

